System Design — Chapter 1

Scale From Zero to Millions of Users

Revision notes based on our Chapter 1 study session and the uploaded Alex Xu book. The notes preserve the chapter's progression and add the examples, trade-offs, and interview reasoning we discussed.

> **Core mindset:** Don't memorize a giant architecture. Start simple, identify the bottleneck, and evolve the architecture by adding the component that solves that specific problem.

```
Single Server
↓ traffic / availability problem
Multiple Servers + Load Balancer
↓ database SPOF / read bottleneck
Database Replication
↓ repeated expensive reads
Cache
↓ static-content delivery / geographic latency
CDN
↓ state trapped in individual web servers
Stateless Web Tier
↓ global availability / latency
Multiple Data Centers
↓ tight coupling / bursty asynchronous work
Message Queue
↓ database size / write workload
Sharding
```

Chapter 1 Quick Map

Problem	Solution	Main benefit
One server cannot handle growing traffic	Horizontal scaling + load balancer	Capacity + failover
Single DB / read bottleneck	Replication	Read scaling + availability
Repeated DB reads	Application cache	Lower latency + DB load
Static assets / global delivery	CDN	Faster static-content delivery
Session stored in one web server	Stateless web tier	Easy horizontal scaling
Global users / DC outage	Multiple data centers	Availability + lower latency
Tight coupling / slow downstream work	Message queue	Decoupling + buffering + async processing
Huge/write-heavy database	Sharding	Distribute data/workload

1. Single Server Setup

Start with everything running on one server: web application, business logic, storage/cache, etc. The basic request flow is: DNS resolves the domain to an IP, the client sends an HTTP request to the server, and the server returns HTML or JSON.

```
User / Mobile App
|
v
DNS
| domain → IP
v
Single Web Server
|
+--> application logic
+--> database
+--> cache / other components
```

Trade-off: Extremely simple, but the server becomes a single point of failure and has a hard capacity limit.

Interview cue: Start simple. Do not introduce distributed components before identifying a problem that requires them.

2. Database Separation

As the user base grows, separate the web/mobile traffic tier from the data tier so they can be scaled independently. The book discusses both relational databases (such as PostgreSQL/MySQL/Oracle) and NoSQL categories.

```
Users
|
v
Web Tier ---> Data Tier
|
Database
```

SQL vs NoSQL: Relational databases are a strong default for relational data and joins. NoSQL may fit use cases involving very low latency, unstructured data, serialization-oriented storage, or massive data volumes.

3. Vertical vs Horizontal Scaling

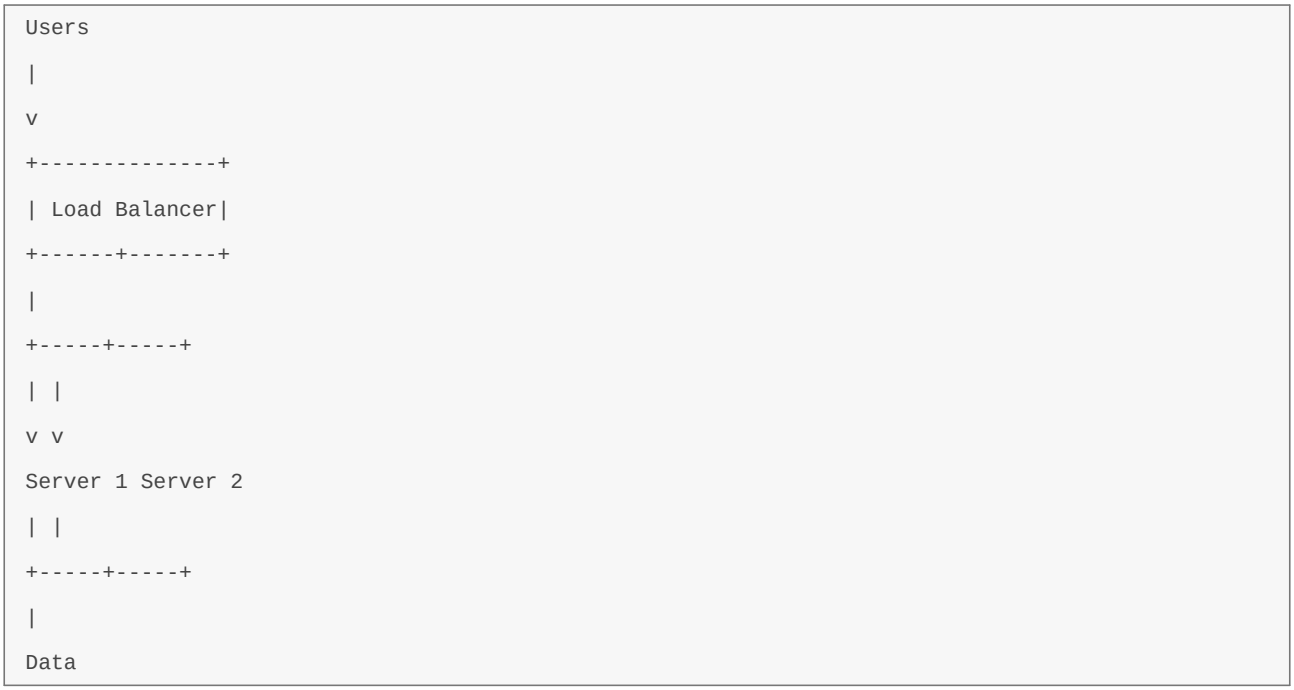
```
Vertical / Scale Up:
[ 4 CPU / 8 GB RAM ] → [ 32 CPU / 128 GB RAM ]

Horizontal / Scale Out:
[ Server 1 ] → [ Server 1 ] [ Server 2 ] [ Server 3 ]
```

Vertical scaling: simple, but has hard limits, can become expensive, and does not inherently provide redundancy.

Horizontal scaling: add servers to a pool. It is more desirable for large-scale systems because capacity and failure handling can be distributed.

4. Load Balancer

A load balancer distributes incoming traffic across a set of web servers. Clients connect to the load balancer's public address; web servers can remain unreachable directly from the public internet and communicate using private IPs.



Benefits: traffic distribution, failover, and easier horizontal scaling. If one server fails, traffic can be routed to healthy servers.

5. Database Replication

The book introduces a master/slave model (modern systems often use primary/replica terminology). Writes and data-modifying operations go to the primary/master; replicas receive copies and can serve reads.

Benefits: better read performance, reliability, and high availability. Reads can be distributed across replicas while writes are handled by the primary.

> **Failover caveat:** if the primary fails, a replica can be promoted, but production failover must account for replication lag and potentially missing data. Recovery and re-synchronization may be required.

Rule: Replication is especially useful for read-heavy workloads. It does not automatically solve a write bottleneck on the primary.

6. Cache

A cache is temporary storage for frequently accessed or expensive-to-compute data. The application checks the cache first; on a miss, it queries the database, stores the result in cache, and returns it.

```
Request
|
v
Cache?
/ \
HIT MISS
| |
v v
Return Database
|
v
Cache
|
v
Return
```

Cache hit: requested data is found in cache. **Cache miss:** data is absent, so the application retrieves it from the database and populates the cache.

Good candidates: data read frequently and modified infrequently. Cache is a performance layer, not the persistent source of truth.

TTL / expiration: too short causes frequent database reloads; too long increases the chance of stale data.

Consistency: database and cache updates are not automatically one atomic transaction. A common application-level strategy is to update the database and invalidate the affected cache entry, with TTL as a safety net.

Eviction: when cache capacity is full, entries are removed. LRU (Least Recently Used) is a common policy; LFU and FIFO are alternatives.

Failure: a single cache server can be a SPOF. Design for cache failure and avoid treating the cache as the only copy of important data.

7. CDN

A CDN is a geographically distributed network used to deliver static content such as images, videos, CSS, and JavaScript. It places cached content closer to users.

```
Origin
Web Server / S3
/ | \
v v v
CDN India CDN USA CDN Europe
^ ^ ^
| | |
Users Users Users
```

CDN cache hit: the edge server already has the object and returns it directly. **CDN cache miss:** the edge fetches the object from the origin, caches it, then returns it.

CDN vs application cache: application caches commonly hold application/database data (e.g., Redis), while the CDN in this chapter primarily caches static assets close to users.

Trade-offs: cost, cache-expiry tuning, CDN failure/fallback, and invalidation. The book mentions explicit invalidation and object versioning such as image.png?v=2.

8. Stateless Web Tier

A stateful web server remembers client state locally. This creates a problem when a load balancer sends subsequent requests to another server. Sticky sessions can force a user back to the same server, but they complicate scaling and failure handling.

```
Stateful:
User A → Load Balancer → Server 1 → Session A stored locally
User A → Load Balancer → Server 1 (must remain sticky)

Stateless:
User A → Load Balancer → Server 1 ■■
User A → Load Balancer → Server 2 ■■→ Shared Store
User A → Load Balancer → Server 3 ■■
```

Stateless design: move session/state out of individual web servers into shared storage such as a relational DB, Redis/Memcached, or NoSQL store. Then any server can handle the request.

> **JWT connection:** a self-contained signed JWT can carry authentication claims so each pod can validate the request independently; Redis is not necessarily required just to maintain an authentication session. JWT contains claims and a signature—not a private key.

9. Multiple Data Centers

For global systems, multiple data centers can improve user experience and availability. GeoDNS can direct users toward an appropriate/nearby data center.

```
Global Users
|
GeoDNS
/ \
v v
Mumbai DC USA DC
| |
Application Application
| |
Cache Cache
| |
Data Data
\ /
replication
```

New challenges: traffic routing, data synchronization/replication, and consistent deployment across data centers. A data-center outage should not take down the entire service.

10. Message Queue

A message queue decouples producers and consumers. A producer publishes a message; the queue buffers it; consumers process messages asynchronously.

```
Producer
|
v
+-------------------------+
| Queue: M1 | M2 | M3 ... |
+-------------------------+
| |
v v
Consumer 1 Consumer 2
```

Why use it? Decoupling, independent scaling, buffering of traffic spikes, and asynchronous processing. For example, loan approval can complete without making the user wait for a slow email provider.

Scaling consumers: add consumer instances so work can be processed in parallel. In Kafka specifically, parallelism within a consumer group is constrained by partitions.

Delivery/retry issue: a consumer can process a message, perform an external action, then crash before acknowledging/committing. The message may be delivered again. This makes idempotency important.

> **Idempotent consumer pattern:** use a unique event ID and record processed IDs so duplicate deliveries can be detected. However, an external action such as email plus a DB update/offset commit is not automatically one atomic transaction, so failure windows still need careful design.

11. Database Scaling & Sharding

Vertical database scaling increases CPU/RAM on one machine but has hard limits, cost, and availability concerns. Replication helps read scaling, but a write-heavy primary can still be overloaded.

```
Replication = copies of the same data:
Primary → Replica 1 / Replica 2

Sharding = split data across databases:
Data
/ | \
Shard 1 Shard 2 Shard 3
```

Sharding: partition the dataset across multiple database shards so storage and workload are distributed. A shard key determines where a record belongs.

Example: userId can be used conceptually as a shard key: hash(userId) → shard. A good shard key should support common access patterns and distribute data/workload reasonably evenly.

Bad shard key example: country when 70% of users are from India can create a hot shard. The purpose of sharding is undermined if one shard receives most of the traffic.

Sharding trade-offs: cross-shard queries/joins become harder; cross-shard transactions become more complex; resharding can require significant data movement; poor shard-key selection creates hotspots.

Replication + sharding: they solve different problems and can be combined. Sharding distributes data/workload; replication provides redundancy and read scaling for each shard.

```
Database Cluster
/ | \
```

```
Shard 1 Shard 2 Shard 3
| | |
Primary Primary Primary
| | |
Replica Replica Replica
```

12. Final Chapter 1 Architecture

The final architecture is an evolution, not a mandatory checklist. Each component is introduced because a particular bottleneck or reliability problem appears.

```
USERS
|
DNS / GeoDNS
|
+-------------+-------------+
| |
CDN Load Balancer
| |
Static Content Stateless Web Tier
|
Cache
|
Message Queue
|
Consumers
|
+-----------+-----------+
| | |
Shard 1 Shard 2 Shard 3
| | |
Primary Primary Primary
| | |
Replica Replica Replica
```

Remember: a real design may omit components or use different technologies depending on requirements. The goal is to explain why a component is needed, its trade-offs, and how the architecture evolves.

13. Interview Cheat Sheet

Question: Traffic increases 10×. What do you do?
First identify the bottleneck and read/write ratio. Read-heavy database load may benefit from replicas; write-heavy load may require other scaling approaches such as sharding.

Question: Cache contains old loan status.
Recognize stale data. Invalidate the affected cache entry when the DB changes and use TTL as a safety net. Consider failure windows because DB update and cache invalidation are not automatically atomic.

Question: Pod 1 crashes and its session was in memory.
The session is lost to the remaining pods. Move session state to shared storage, or use a stateless authentication approach such as self-contained signed JWTs when appropriate.

Question: Email provider takes 5 seconds.
Decouple notification with asynchronous messaging so the user-facing operation doesn't need to wait for the email provider.

Question: Consumer receives the same message twice.
Design the consumer to be idempotent, commonly using a unique event ID and a processed-event record. Do not assume offsets alone make an external side effect exactly-once.

Question: Consumer cannot keep up with producers.
Scale consumers horizontally. With Kafka, partition count limits the number of active consumers in a consumer group.

Question: Database primary is write-overloaded.
Read replicas won't remove the primary's write bottleneck. Consider sharding with a carefully selected shard key.

Question: One shard is hot.
The shard key is producing uneven data or traffic distribution. Re-evaluate the key and distribution strategy.

14. Final Takeaways

1. **Start simple.** Begin with a single server and evolve only when a real constraint appears.

2. **Scale the bottleneck.** A 10× traffic increase does not automatically mean adding every distributed component.

3. **Know the distinction:** replication creates copies; sharding splits data.

4. **Cache is an optimization.** Database remains the persistent source of truth in the basic model.

5. **Stateless web tiers scale cleanly.** Keep request/session state outside individual application instances when possible.

6. **Queues decouple and buffer.** They also introduce retries, duplicate delivery, and idempotency concerns.

7. **Sharding is powerful but expensive operationally.** Shard-key selection is one of the most important decisions.

8. **Architecture is a trade-off.** Every new component solves a problem but introduces new failure modes and complexity.

> **Chapter 1 objective:** Be able to draw the architecture and explain, component by component, *what problem it solves, why it helps, what can go wrong, and what trade-off it introduces.*

Source: Alex Xu, *System Design Interview: An Insider's Guide*, uploaded book. These notes are a study/revision aid based on the book and our discussion; they are not a replacement for the original text.